

# The Question Everyone Asks

Is "e" a physical table or not?

---

When you write a query like this:

```
FROM employees e
  LEFT JOIN departments d ON e.department_id = d.department_id
  LEFT JOIN jobs j ON e.job_id = j.job_id
  LEFT JOIN locations l ON d.location_id = l.location_id
```

People ask: "The 3rd JOIN says d.location\_id. But at that point, we are working on the result of 2 JOINs. So where is departments? Is d still a physical table?"

**Answer: NO. After the first JOIN, the physical employees table and the physical departments table are GONE. A brand new virtual table is created. The aliases "e" and "d" are just column name labels inside this new virtual table.**

Let us understand this step by step using a spreadsheet analogy.

# Think of It as Excel Sheets

Imagine you are working in Excel. You have two separate files:

**File 1: employees.xlsx**  
Physical table on your hard drive

| e.first_name | e.last_name | e.salary | e.job_id | e.department_id |
|--------------|-------------|----------|----------|-----------------|
| Alice        | Smith       | 5000     | IT_PROG  | 60              |

This is a real file. Columns are labeled "e." because the SQL alias is "e".

**File 2: departments.xlsx**  
Another physical table

| d.department_id | d.department_name | d.location_id |
|-----------------|-------------------|---------------|
| 60              | IT                | 1500          |

Another real file. Columns are labeled "d." because the SQL alias is "d".

## Now the JOIN happens...

When SQL executes `LEFT JOIN departments d ON e.department_id = d.department_id`, it does something very specific:

Step A: It reads both files and matches rows by `department_id`.

Step B: It creates a brand new combined sheet with columns from both files.

Step C: The original two files are no longer needed. SQL now works only with the new sheet.

This new combined sheet is the Virtual Table. It is NOT `employees.xlsx` and it is NOT `departments.xlsx`. It is a brand new thing. But the column labels "e." and "d." are kept so you know where each column came from.

## What the New Virtual Table Looks Like

After the first JOIN, a brand new virtual table is created. It contains ALL columns from both tables. The "e." and "d." prefixes are just labels to track the origin:

| VIRTUAL TABLE #1                                              |  |  |  |  |  |  |  |
|---------------------------------------------------------------|--|--|--|--|--|--|--|
| Created by JOIN #1. Neither File 1 nor File 2 exists anymore. |  |  |  |  |  |  |  |

| e.first_name | e.last_name | e.salary | e.job_id | e.department_id | d.department_id | d.department_name | d.location_id |
|--------------|-------------|----------|----------|-----------------|-----------------|-------------------|---------------|
| Alice        | Smith       | 5000     | IT_PROG  | 60              | 60              | IT                | 1500          |

Blue columns came from "e" (employees). Green columns came from "d" (departments). But this is ONE virtual table now. Not two separate tables.

## Now the 2nd JOIN happens...

SQL executes: LEFT JOIN jobs j ON e.job\_id = j.job\_id

Where does "e.job\_id" come from? It comes from this virtual table (Virtual Table #1), NOT from the original employees file. Look at the table above - there it is, the 4th column, labeled "e.job\_id". SQL reads it from here.

Then SQL takes jobs data, matches it, and creates ANOTHER brand new virtual table:

| VIRTUAL TABLE #2                                       |  |  |  |  |  |  |  |
|--------------------------------------------------------|--|--|--|--|--|--|--|
| Created by JOIN #2. Virtual Table #1 no longer exists. |  |  |  |  |  |  |  |

| e.first_name | e.last_name | e.salary | e.job_id | e.dept_id | d.dept_name | d.location_id | j.job_id | j.job_title |
|--------------|-------------|----------|----------|-----------|-------------|---------------|----------|-------------|
| Alice        | Smith       | 5000     | IT_PROG  | 60        | IT          | 1500          | IT_PROG  | Programmer  |

Now this is the only table that exists. Virtual Table #1 is gone. But "e.job\_id" is still here (column 4) because it was copied from VT#1.

# The Key Moment: 3rd JOIN

Now SQL executes: LEFT JOIN locations l ON d.location\_id = l.location\_id

This is where people get confused. They ask: "But d.location\_id... we are not on the departments table anymore. We just did 2 JOINS. So where is the departments table?"

The departments table does NOT exist anymore. It was merged into Virtual Table #1, which was then merged into Virtual Table #2. The column "d.location\_id" is sitting right there in Virtual Table #2 (column 7, see previous page). SQL reads it from there.

Think of it like making a smoothie:

| Step | What You Do                                                                                    | What Exists Now                                                                    |
|------|------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------|
| 1    | Take apple (e) alone                                                                           | Bowl: apple                                                                        |
| 2    | Add banana (d) to bowl                                                                         | Bowl: apple + banana<br>(bowl is now ONE thing, not "apple" + "banana" separately) |
| 3    | Add strawberry (j) to bowl                                                                     | Bowl: apple + banana + strawberry<br>(still ONE bowl)                              |
| 4    | Add blueberry (l) using "banana flavor"<br>(BUT the banana is PART of the bowl, not separate!) | Bowl: apple + banana + strawberry + blueberry<br>(ONE bowl with 4 fruits)          |

At Step 4, you do NOT go to the original banana. You taste the banana flavor from the bowl (which already has it mixed in).

# The Complete Lifecycle

Here is what actually exists in memory at each step. Notice how the old virtual table disappears and a new one is created:

## Before Any JOIN:

**employees table (physical)**  
This is a real table in the database

Only this exists. No virtual tables yet. Alias "e" points to this physical table.



**JOIN #1: employees + departments**



## After JOIN #1:

**VIRTUAL TABLE #1**  
employees + departments merged into one

The physical employees table is GONE from memory. The physical departments table is GONE from memory. Only Virtual Table #1 exists. It has columns labeled "e.xxx" and "d.xxx" but those are just name prefixes inside this one table. There is no "e" table or "d" table anymore. There is only VT#1.



**JOIN #2: VT#1 + jobs**



## After JOIN #2:

**VIRTUAL TABLE #2**  
VT#1 + jobs merged into one

Virtual Table #1 is GONE. Only VT#2 exists now. It has columns "e.xxx", "d.xxx", and "j.xxx" - all just labels inside this single table. When you write "e.job\_id", SQL looks inside VT#2 and finds the column labeled "e.job\_id". It does NOT go to the original employees table (that table is long gone from memory).



**JOIN #3: VT#2 + locations → ON d.location\_id = l.location\_id**



## After JOIN #3:

### **VIRTUAL TABLE #3**

**VT#2 + locations merged into one**

"d.location\_id" comes from VT#2 (column 7 from the previous page). VT#2 had that column because it was copied from VT#1, which got it from the original departments table. The "d." prefix is just a name tag. SQL scans VT#2, finds the column tagged "d.location\_id", and uses its value (1500). Then VT#2 disappears and VT#3 is created with location columns added.

# The Rules

Rule 1: Each JOIN creates a NEW virtual table.

After the JOIN is done, the previous virtual table is destroyed. The new virtual table contains ALL columns from the previous one plus the new table.

Rule 2: "e", "d", "j" etc. are NOT tables.

They are alias prefixes. After the first JOIN, there is no "employees" table and no "departments" table. There is one virtual table with columns labeled "e.first\_name", "d.department\_name" etc. The alias is just part of the column name.

Rule 3: When you write "e.job\_id" in the 3rd JOIN, SQL looks inside the CURRENT virtual table.

It does NOT open the original employees table. It scans the columns of the current virtual table (VT#2) and finds the one labeled "e.job\_id". That column was copied from VT#1, which got it from the original table. But the original table is gone.

Rule 4: "d.location\_id" in the 4th JOIN = the d.location\_id column inside the CURRENT virtual table.

By the time we reach the 4th JOIN (locations), we are on Virtual Table #3. This table has a column labeled "d.location\_id" because it was inherited from VT#2 (which got it from VT#1, which got it from the departments table). SQL reads it from VT#3. The original departments table has been irrelevant since JOIN #1 finished.

Rule 5: You can only use aliases that have already been joined.

In the 3rd JOIN, you can use "e" and "d" because they were joined in steps 1 and 2. You CANNOT use "l" (locations) because it has not been joined yet. The alias becomes available only AFTER its table is joined.

# One Picture Summary

Here is the complete lifecycle in one view. Follow the arrows:

| Step                                             | What SQL Does                                        | What Exists in Memory    | Available Aliases |
|--------------------------------------------------|------------------------------------------------------|--------------------------|-------------------|
| FROM employees e                                 | Load employees table                                 | employees (physical)     | e                 |
| LEFT JOIN dept d ON ...                          | Merge + create VT#1                                  | VT#1 (e + d columns)     | e, d              |
| LEFT JOIN jobs j ON ...                          | Merge + create VT#2                                  | VT#2 (e + d + j columns) | e, d, j           |
| LEFT JOIN loc l ON d.location_id = l.location_id | Read d.location_id from VT#2,<br>merge + create VT#3 | VT#3 (e+d+j+l columns)   | e, d, j, l        |
| LEFT JOIN countries c ON ...                     | Read l.country_id from VT#3,<br>merge + create VT#4  | VT#4                     | e, d, j, l, c     |
| LEFT JOIN regions r ON ...                       | Read c.region_id from VT#4,<br>merge + create VT#5   | VT#5 (final result)      | e, d, j, l, c, r  |

Orange row = the d.location\_id moment. It reads from VT#2, not the original departments table.

**Every JOIN = Create a new virtual table, destroy the old one.**  
**Aliases = Column name labels inside the current virtual table.**  
**"e", "d", "j" are NOT tables. They are prefixes like "first\_name" or "salary".**